Modernizing .NET

Native AOT and Sna

SVS344 · Doug Perkes · Lass



1097

1097

p50 cold start. ASP.NET Core AP

Today

320

p50 cold start. ASP.NET Core AP

Today

320

p50 cold start. ASP.NET Core AP

Two paths. Neither is flip

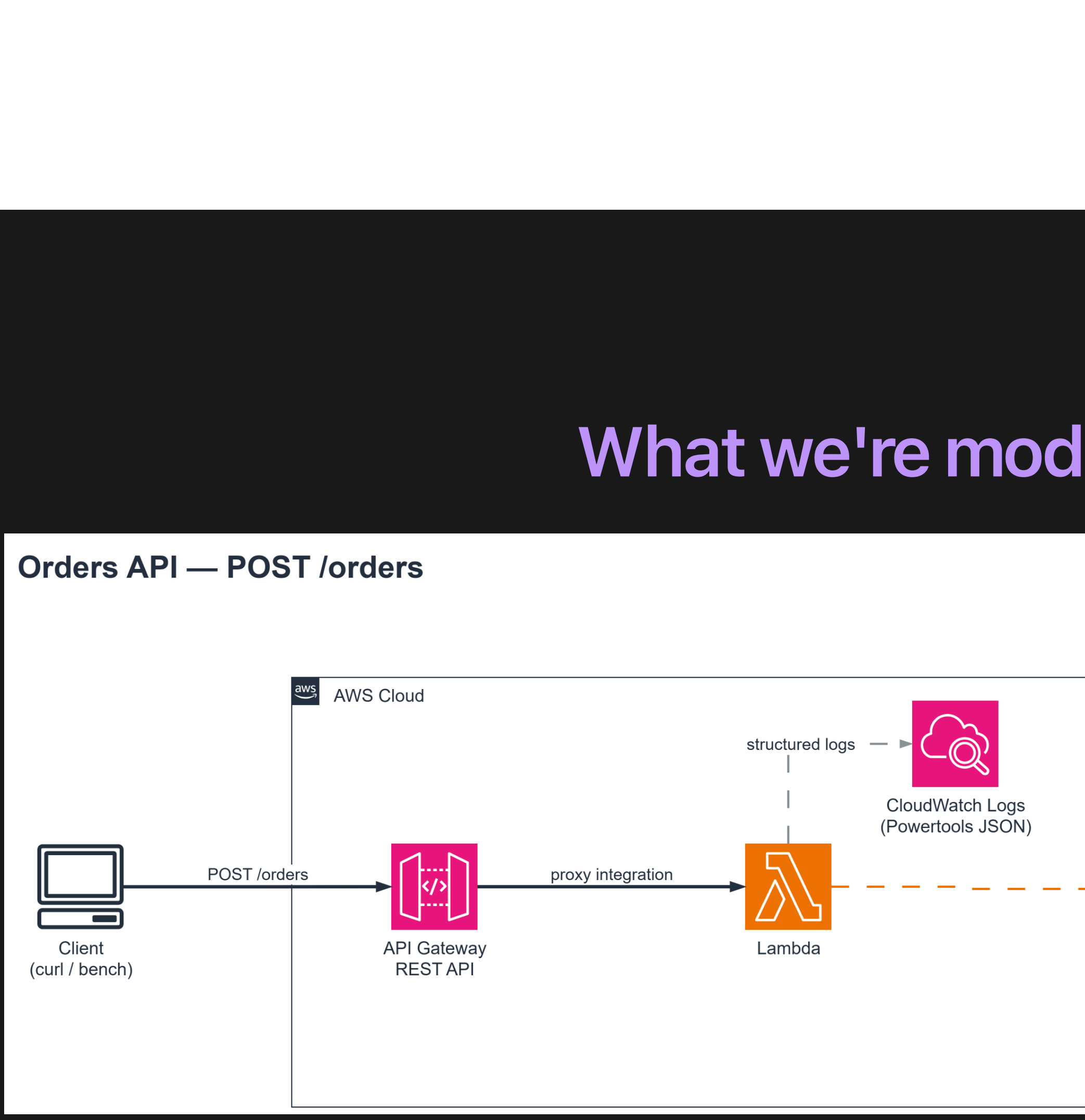
Grab the repo



s12d.com/riv2

Every failure. Every

What we're mod



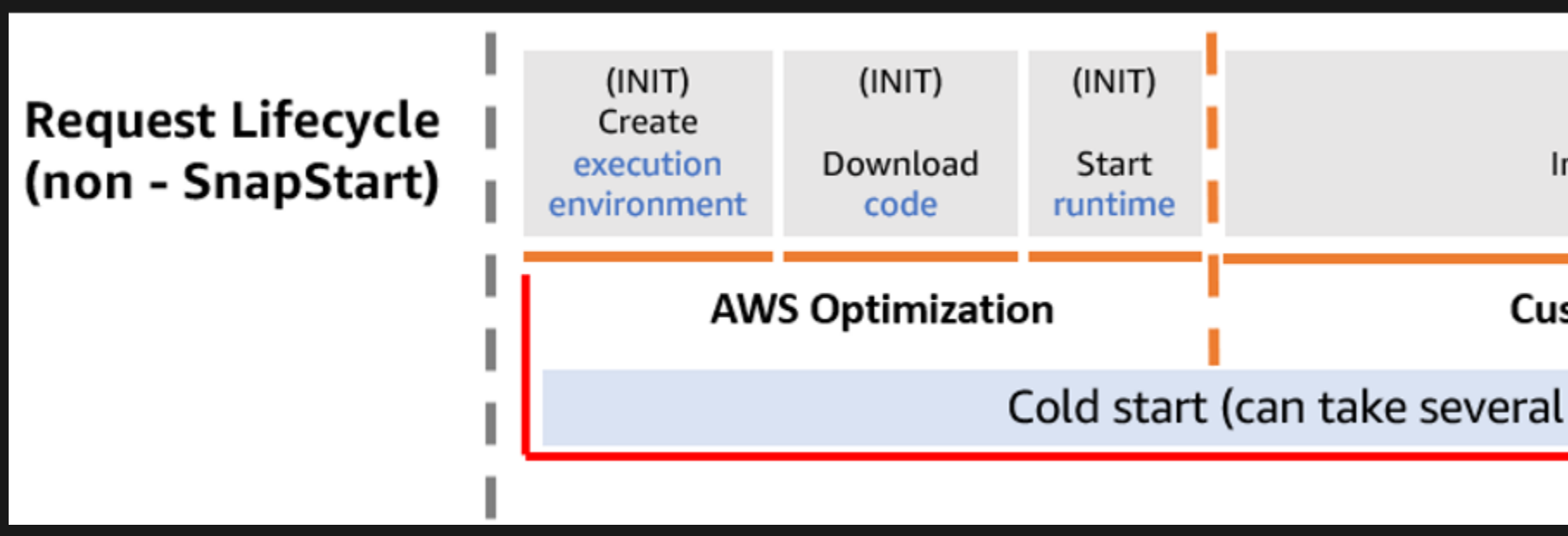
Same code. Three

Same in all three
Runtime: dotnet10
Architecture: arm64
Memory: 2048 MB
Region: us-east-1

Managed

01-managed/

Where that tim

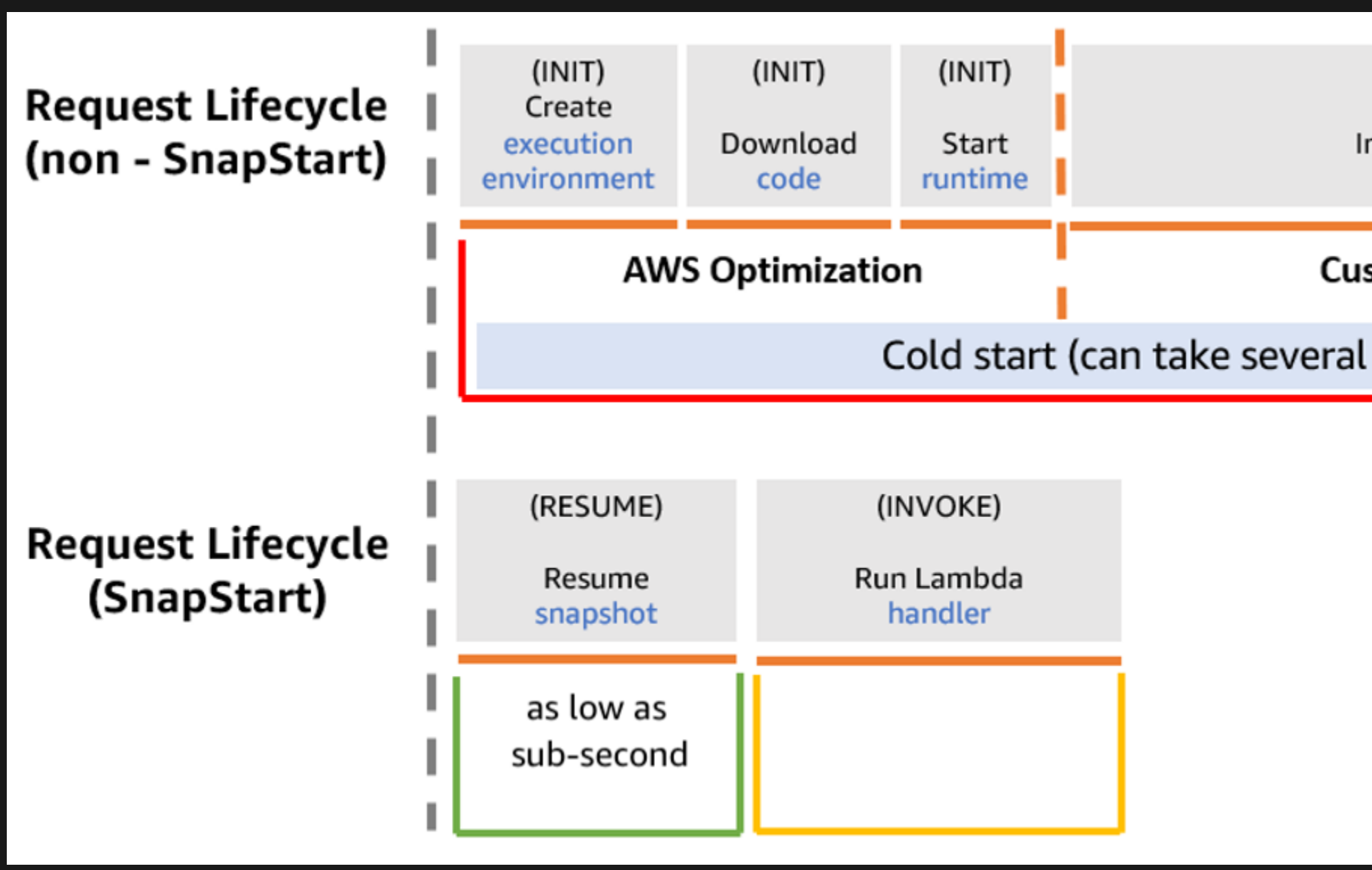


For .NET, **JIT** is the expensive part — during in

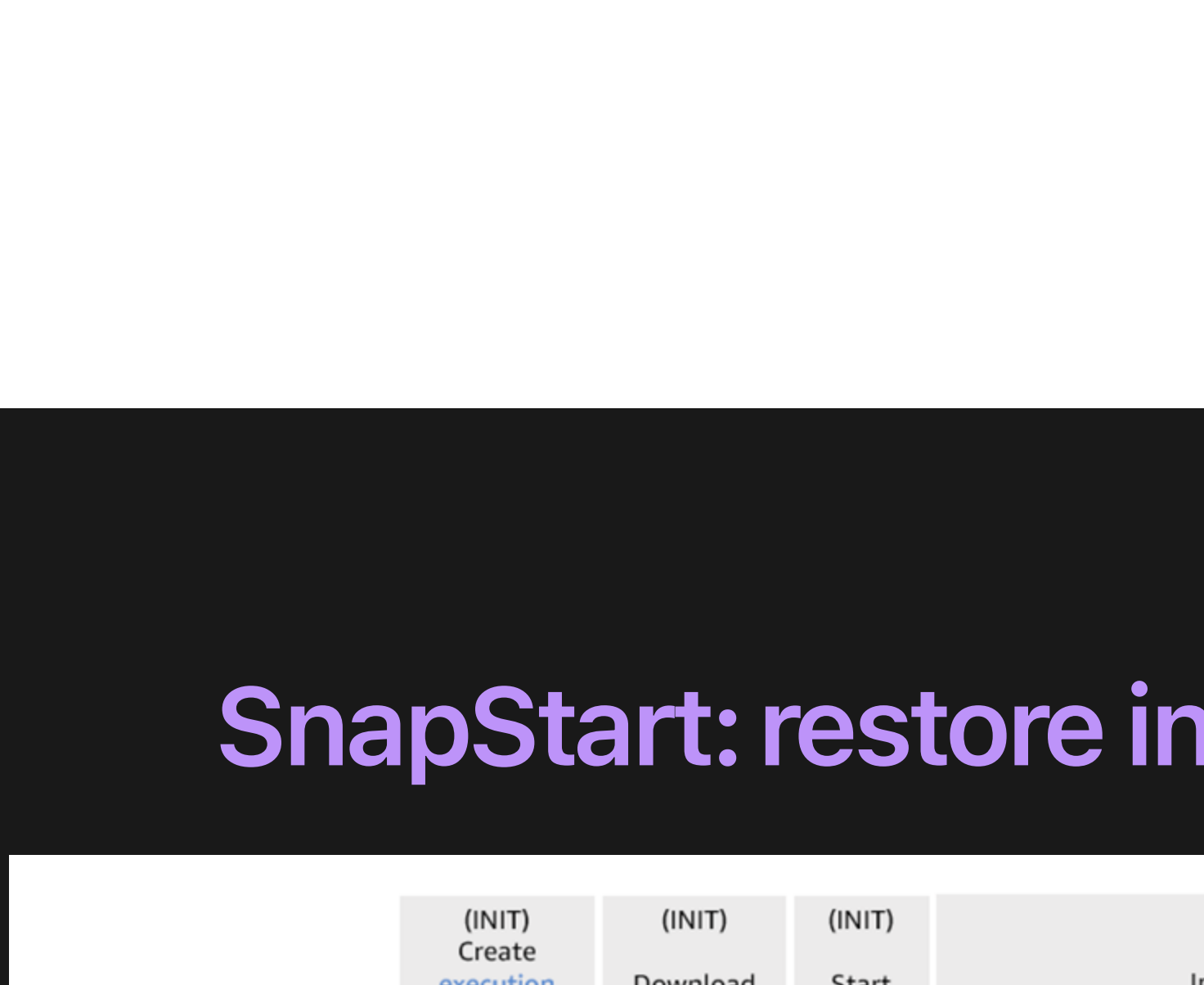
SnapStart

02-snapstart/

Same request, two

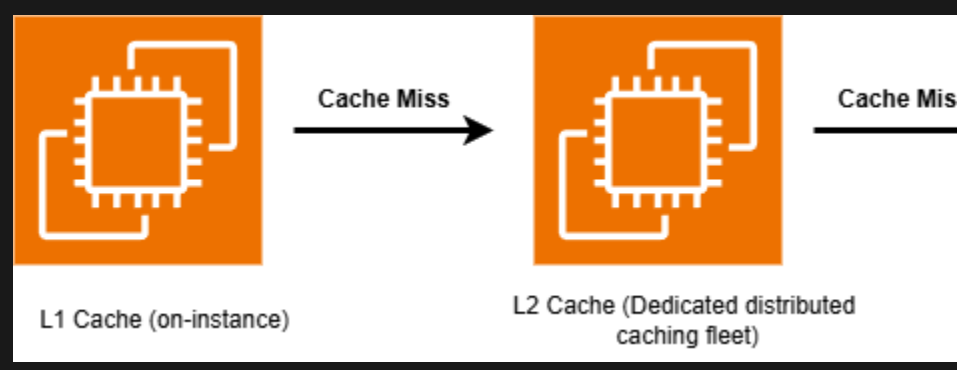


SnapStart: restore in



Init runs once, at **version**

Where the snaps



~1ms L1 · **single-digit ms** L2 · **h**

Enabling it is not

	Init / restore duration (p50)
Managed	394ms *(init)*
SnapStart	647ms *(restore)*
SnapStart primed	718ms *(restore)*

Priming

Run the real path **before** the snapshot. F

Iteration	Run
1	447ms
2	8.7ms
3–10	3.4–6.0

Provisional — single publish. Final numbers

What the snapsh

Random seeds — re-seed

Connections — release before

`DateTime.UtcNow` at init — ne

Native AOT

`03-native-aot/`

1. Serialization
2. Dependency injection
3. Trimming warnings

Head to he

	Managed	SnapSta
Cold total p50	1097ms	952ms
Cold total p99	1637ms	1248ms
Init / restore p50	394ms	718ms
Duration p99	828ms	309ms
Warm avg	11.7ms	13.1ms
Max memory	133MB	134MB
Effort	none	low

Which pat

Latency-critical, willing to in

EF Core or heavy reflection

Cold starts don't matter → ma

Take these wit

Library compatibility matrix — che

Modernization patterns — the

Monday mor

1. Measure your cold start. Kn
2. Enable SnapStart on one fu
3. Pick one hot path, check th

Questio

s12d.com/riv2

Please fill out the sessio